0127. Word Ladder

ulty: Hard | Category: Graphs | LeetCode: https://leetcode.com/problems/word-la

Problem

- Transform beginWord to endWord changing one letter at a time; every intermediate word must be in wordList. Return the number of words in the shortest sequence, or 0.
- Example: hit → hot → dot → dog → cog = 5.

Intuition

- Words are nodes; edges connect words differing by one letter. Shortest path in an unweighted graph = BFS.
- Generate neighbours by trying all 26 letters at each position (L 26 candidates) instead of comparing with every word.

Brute Force

- Build the full graph by comparing all word pairs: O(N^2 L).

Optimal Approach

- Dict = set(wordList); if endWord missing return 0.
- BFS from beginWord with level = 1.
- For each word, for each position and letter: if the candidate is in dict, remove it (marks visited) and enqueue; if it equals endWord return level + 1.

Worked Example

```
hit -> cog, list [hot,dot,dog,lot,log,cog].

level 1: hit
level 2: hot
level 3: dot, lot
level 4: dog, log
level 5: cog -> return 5
```

Java Solution

```java
public class WordLadder {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> dict = new HashSet<>(wordList);
        if (!dict.contains(endWord)) return 0;
        Deque<String> q = new ArrayDeque<>(List.of(beginWord));
        dict.remove(beginWord);
        for (int level = 1; !q.isEmpty(); level++) {
            for (int i = q.size(); i > 0; i--) {
                char[] w = q.poll().toCharArray();
                for (int p = 0; p < w.length; p++) {
                    char orig = w[p];
                    for (char c = 'a'; c <= 'z'; c++) {
                        if (c == orig) continue;
                        w[p] = c;
                        String cand = new String(w);
                        if (dict.remove(cand)) {
                            if (cand.equals(endWord)) return level + 1;
                            q.add(cand);
                        }
                    }
                    w[p] = orig;
                }
            }
        }
        return 0;
    }
}
```

Complexity

- Time: O(N L 26). Space: O(N L).

Pitfalls

- Remove words from the set when enqueued, not when dequeued, to avoid duplicates in the queue.
- Bidirectional BFS is a common follow-up speed-up.